Introduction to Machine Learning
Problems: Principal Component Analysis (with Solutions)

Profs. Sundeep Rangan and Yao Wang

1. Assume that you have 4 samples each with dimension 3, described in the data matrix X,

$$X = \begin{bmatrix} 3 & 2 & 1 \\ 2 & 4 & 5 \\ 1 & 2 & 3 \\ 0 & 2 & 5 \end{bmatrix}$$

For the problems below, you may do the calculations in python. Describe the commands you used.

(a) Find the sample mean.

(b) Find the sample covariance matrix Q.

(c) Find the eigenvalues and eigenvectors. You can use the numpy.linalg.eig to compute eigenvalues and eigenvectors from Q.

(d) Find the PCA coefficients corresponding to samples in X.

(e) Reconstruct the original samples from the PCA coefficients.

(f) Approximate the samples using principle components corresponding to the two largest eigenvalues.

(g) Verify the sum of reconstruction error squares = sum of squares of skipped PCA coefficients.

Solution:

(a) The sample mean is computed as:

```
import numpy as np
X = np.array([[3, 2, 1], [2, 4, 5], [1, 2, 3], [0, 2, 5]])
mu = np.mean(X, axis=0)
```

The mean is $\boldsymbol{\mu} = [1.5, 2.5, 3.5]$.

(b) The sample covariance matrix is computed as:

```
X_centered = X - mu
Q = np.cov(X_centered.T)
# Or equivalently: Q = (1/(n-1)) * X_centered.T @ X_centered
```

The covariance matrix is:

$$Q = \begin{bmatrix} 2.0 & 0.0 & -1.0 \\ 0.0 & 1.0 & 0.0 \\ -1.0 & 0.0 & 2.0 \end{bmatrix}$$

(c) Eigenvalues and eigenvectors:

```
eigenvals, eigenvecs = np.linalg.eig(Q)
# Sort by eigenvalues in descending order
idx = eigenvals.argsort()[::-1]
eigenvals = eigenvals[idx]
eigenvecs = eigenvecs[:, idx]
```

The eigenvalues are approximately $\lambda_1 = 3.0$, $\lambda_2 = 2.0$, $\lambda_3 = 0.0$. The corresponding eigenvectors (principal components) are the columns of the eigenvector matrix.

(d) PCA coefficients are computed by projecting centered data onto the principal components:

```
V = eigenvecs  # Principal components (columns)
Z = X_centered @ V  # PCA coefficients
```

Each row of Z contains the PCA coefficients for the corresponding sample.

(e) Reconstruction from PCA coefficients:

```
X_reconstructed = Z @ V.T + mu
```

This should recover the original X exactly (within numerical precision).

(f) Approximation using two largest PCs:

```
V2 = eigenvecs[:, :2]  # First two principal components
Z2 = X_centered @ V2  # Coefficients for first two PCs
X_approx = Z2 @ V2.T + mu  # Approximation
```

(g) Verification:

```
reconstruction_error = X - X_approx
error_squared_sum = np.sum(reconstruction_error**2)

skipped_coeffs = Z[:, 2:]  # Skipped PCA coefficients
skipped_squared_sum = np.sum(skipped_coeffs**2)

# These should be equal (or very close)
```

The sum of squares of reconstruction errors equals the sum of squares of the skipped PCA coefficients because the reconstruction error is exactly the projection onto the discarded principal components.

2. *PC bases.* Suppose that the mean and first two PCs in a basis are,

$$\boldsymbol{\mu} = [1, 0, 2], \quad \mathbf{v}_1 = \frac{1}{\sqrt{2}}[1, 1, 0], \quad \mathbf{v}_2 = \frac{1}{\sqrt{2}}[1, -1, 0],$$

(a) What are the two PC coefficients of the vector $\mathbf{x} = [2, 3, 4]$?

(b) What is $\widehat{\mathbf{x}}$, the approximation of $\mathbf{x}$ using two PCs?

(c) What is the approximation error $\|\mathbf{x} - \widehat{\mathbf{x}}\|^2$?

Solution:

(a) The PC coefficients are computed by projecting the centered vector onto the PCs:

$$\mathbf{x}_{\text{centered}} = \mathbf{x} - \boldsymbol{\mu} = [2, 3, 4] - [1, 0, 2] = [1, 3, 2]$$

The coefficients are:

$$z_1 = \mathbf{v}_1^T(\mathbf{x} - \boldsymbol{\mu}) = \frac{1}{\sqrt{2}}[1, 1, 0]^T[1, 3, 2] = \frac{1}{\sqrt{2}}(1 + 3) = \frac{4}{\sqrt{2}} = 2\sqrt{2}$$
$$z_2 = \mathbf{v}_2^T(\mathbf{x} - \boldsymbol{\mu}) = \frac{1}{\sqrt{2}}[1, -1, 0]^T[1, 3, 2] = \frac{1}{\sqrt{2}}(1 - 3) = \frac{-2}{\sqrt{2}} = -\sqrt{2}$$

(b) The approximation is:

$$\widehat{\mathbf{x}} = \boldsymbol{\mu} + z_1\mathbf{v}_1 + z_2\mathbf{v}_2 = [1, 0, 2] + 2\sqrt{2} \cdot \frac{1}{\sqrt{2}}[1, 1, 0] + (-\sqrt{2}) \cdot \frac{1}{\sqrt{2}}[1, -1, 0]$$

$$= [1, 0, 2] + 2[1, 1, 0] - [1, -1, 0] = [1, 0, 2] + [2, 2, 0] - [1, -1, 0] = [2, 3, 2]$$

(c) The approximation error is:

$$\|\mathbf{x} - \widehat{\mathbf{x}}\|^2 = \|[2, 3, 4] - [2, 3, 2]\|^2 = \|[0, 0, 2]\|^2 = 4$$

Note that this error equals the square of the third (skipped) PC coefficient, which would be the projection onto the direction perpendicular to the first two PCs.

3. *Fitting data with PCs.* You are given python functions for PCA transform and a binary classifier:

```
mu, V = PCA(X)  # Finds the mean and PCs for a data matrix X

clf = Classifier()
clf.fit(Z,y)    # Fits a binary classifier from features Z and labels y
yhat = clf.predict(Z)  # Predicts labels from Z
```

Given a data matrix `X` with binary labels `y`, you wish to build a classifier that uses a PCA transform followed by the `Classifier`. Write python code that uses model validation to select the optimal number of PCA components to use. You may assume:

- You use simple cross validation with one training and test split. Not You do not need to do K-fold validation.

- You should use roughly 25% of the samples for test. Data shuffling before splitting is not needed.

Solution:

```
n = len(X)
n_test = int(0.25 * n)
n_train = n - n_test

# Split data
X_train = X[:n_train]
X_test = X[n_test:]
y_train = y[:n_train]
y_test = y[n_test:]

# Find PCA on training data
mu, V = PCA(X_train)

# Try different numbers of components
best_nc = 0
best_acc = 0

for nc in range(1, min(n_train, X.shape[1]) + 1):
    # Transform training data
    X_train_centered = X_train - mu
    Z_train = X_train_centered @ V[:, :nc]

    # Fit classifier
    clf = Classifier()
    clf.fit(Z_train, y_train)

    # Transform test data
    X_test_centered = X_test - mu
    Z_test = X_test_centered @ V[:, :nc]

    # Predict and evaluate
    yhat_test = clf.predict(Z_test)
    acc = np.mean(yhat_test == y_test)

    if acc > best_acc:
        best_acc = acc
        best_nc = nc

print(f"Best number of components: {best_nc}")
```

4. *PC with images.* A dataset has 1000 images of size 28×28, represented as python array X

with shape (1000,28,28). You are given the following python functions:

```
Y = reshape(X, shape)          # Reshapes X to a shape
pca = PCA(n_components = nc)  # Creates a PCA object
pca.fit(Y)    # Finds the mean and PCs from training data Y
Z = pca.transform(Y)    # Find coefficients in the PC basis
Yhat = pca.inverse_transform(Z)  # Invert the PC transform
```

Write a few lines of python code to (i) fit a PC model from the first 500 images with 5 components; and (ii) create an array of approximations of the remaining 500 images.

Solution:

```
# (i) Fit PCA model from first 500 images
X_train = X[:500]
X_train_flat = reshape(X_train, (500, 28*28))
pca = PCA(n_components=5)
pca.fit(X_train_flat)

# (ii) Create approximations of remaining 500 images
X_test = X[500:]
X_test_flat = reshape(X_test, (500, 28*28))
Z_test = pca.transform(X_test_flat)
X_test_approx_flat = pca.inverse_transform(Z_test)
X_test_approx = reshape(X_test_approx_flat, (500, 28, 28))
```

5. *PCs using SVDs.* You are given a python function:

```
U,s, Vtr = svd(Z, full_matrices = False)
```

which computes an "economy" SVD. Given a data matrix X, write python code that:

(i) Finds the PCs and mean of the data.

(ii) Finds the minimum number of PCs in order that the proportion of variance is at least 90%.

(iii) Create an approximation Xhat of X using the number of PCs in part (ii).

Solution:

```
# (i) Find PCs and mean
mu = np.mean(X, axis=0)
X_centered = X - mu
U, s, Vtr = svd(X_centered, full_matrices=False)
# PCs are the columns of Vtr (or rows of Vtr.T)
V = Vtr.T
# Eigenvalues are s^2 / (n-1)
eigenvals = s**2 / (len(X) - 1)
```

```
# (ii) Find minimum number of PCs for 90% variance
cumulative_var = np.cumsum(eigenvals) / np.sum(eigenvals)
nc = np.argmax(cumulative_var >= 0.90) + 1

# (iii) Create approximation
Z = X_centered @ V[:, :nc]
Xhat = Z @ V[:, :nc].T + mu
```